

Aula 07 - Primeira classe Conta

Contexto de negócio

Agora o sistema já opera várias contas, mas o custo de coordenação subiu.

Arrays paralelos funcionam, porém deixam a conta espalhada em partes. Como representar uma conta inteira sem depender de convenção de índice?

Tensão operacional

No procedural indexado, cada operação precisa:

- receber dados por parâmetro;
- saber exatamente qual índice usar;
- confiar que todos os arrays continuam alinhados.

Quando isso falha, o programa pode seguir rodando e mesmo assim produzir resultado de negócio errado.

Objetivo técnico da entrega

- Mostrar claramente o limite do modelo anterior.
- Introduzir classe e objeto como resposta de modelagem.
- Demonstrar como métodos de instância reduzem passagem excessiva de parâmetros.

Recurso técnico desta aula

Recurso inserido: a classe **Conta** com atributos e métodos próprios.

Problema que justifica o novo artefato

No modelo anterior, a conta não existe como unidade, apenas como combinação de partes.

Exemplo do problema:

- titular em um array;
- saldo em outro;
- limite em outro;
- status em outro.

Isso obriga o algoritmo a "montar" a conta o tempo todo.

Pergunta central desta virada:

- se **depositar** recebe só números e índices, como ele "sabe" em qual conta real está operando?

Resposta: não sabe por significado de domínio, só por posição estrutural. Esse é o limite que esta aula enfrenta.

Artefato explicado: classe Conta como unidade de domínio

Artefato desta aula: classe `Conta` com estado e comportamento.

- atributos carregam identidade e saldo;
- métodos executam regra diretamente no próprio estado.

Aqui a POO entra para reduzir acoplamento estrutural do procedural indexado.

Camada 1: novo modelo mental da variável

No procedural, variável costuma guardar valor isolado (`saldo`, `limite`, `ativo`).

Aqui começa outro jeito de pensar:

- a variável `conta` não guarda só um número;
- ela referencia uma unidade de domínio completa;
- essa unidade traz dados e também comportamento válido para esses dados.

Analogia útil para sala:

- antes: variáveis soltas, e o "controle" das regras ficava fora;
- agora: `conta` funciona como um pequeno módulo de negócio pronto para operar.

Pergunta de condução:

- quando você lê `conta.depositar(200)`, o foco está no número ou na entidade que recebe a operação?

Camada 2: o que muda no fluxo do algoritmo

No modelo anterior, o fluxo principal precisava coordenar detalhes estruturais:

- escolher arrays corretos;
- escolher índice correto;
- chamar função externa com todos os dados necessários.

Agora o fluxo muda de papel:

- o fluxo principal passa a orquestrar intenções (`depositar`, `sacar`);
- a regra detalhada fica encapsulada dentro da classe.

Resultado prático:

- menos ruído estrutural no ponto de uso;
- maior clareza de leitura para revisão e manutenção.

Camada 3: deslocamento da regra de negócio

Essa aula desloca parte da regra para dentro da classe sem esconder a lógica.

Antes:

- regra de validação e atualização espalhada em funções externas;
- alto acoplamento do chamador com estrutura dos dados.

Depois:

- `depositar` e `sacar` validam e atualizam o estado da própria conta;
- o chamador não precisa "remontar" contexto da conta a cada chamada.

Ponto crítico para fixar:

- não é só mudança de sintaxe;
- é mudança de responsabilidade no desenho do sistema.

Antes e depois da mudança

Antes (procedural por índice):

```
depositar(saldos, 0, 200.00);  
boolean ok = sacar(saldos, limites, ativas, 1, 50.00);
```

Depois (conta como unidade):

```
conta.depositar(200.00);  
boolean ok = conta.sacar(50.00);
```

O que muda de verdade:

- a operação passa a acontecer na própria conta, não em estruturas externas;
- reduzimos a carga mental de "qual array + qual índice";
- fica mais claro evoluir regra sem quebrar chamadas.

Demonstração prática do impacto

Cenário antigo:

- índice errado pode alterar saldo de outra pessoa;
- o compilador não avisa que a lógica de negócio ficou incorreta.

Cenário atual:

- `conta.depositar(...)` deixa explícito o alvo da operação;
- identidade e saldo permanecem juntos na mesma estrutura.

Provocação para a turma:

- qual versão é mais fácil de revisar em code review sob pressão?
- em qual versão é mais fácil explicar um bug para alguém novo no time?
- em qual versão a regra de negócio fica mais auditável?

Micro-demonstração guiada (3 passos):

1. Ler a chamada `depositar(...)` no modelo anterior e listar tudo que o chamador precisa saber.

2. Ler a chamada `conta.depositar(...)` e listar o que ainda é responsabilidade do chamador.
3. Comparar as listas e identificar o que foi transferido para dentro da classe.

Transição didática para a próxima aula

Com uma conta modelada como unidade, o próximo passo é escalar para várias contas mantendo essa mesma clareza de modelagem.

Valor prático entregue

- Diminui acoplamento entre dados e regras.
- A base fica pronta para evolução controlada da próxima capacidade.

Fluxo operacional explicado

- Preparar os dados de entrada do cenário da aula.
- Executar a regra principal e observar o impacto no estado.
- Confirmar saída de sucesso, erro e borda antes de concluir a etapa.

Código em foco

```
class Conta {
    int id;
    String titular;
    double saldo;

    Conta(int p_id, String p_titular, double p_saldoInicial) {
        id = p_id;
        titular = p_titular;
        saldo = p_saldoInicial;
    }

    void depositar(double p_valor) {
        if (p_valor > 0) {
            saldo = saldo + p_valor;
        }
    }

    boolean sacar(double p_valor) {
        if (p_valor > 0 && p_valor <= saldo) {
            saldo = saldo - p_valor;
            return true;
        }
        return false;
    }

    String resumo() {
        return "ID: " + id + " | Titular: " + titular + " | Saldo: " + saldo;
    }
}
```

```
// Classe temporaria de fluxo do programa.
// Neste curso, ela existe para iniciar a execucao em Java.
public class AppGestaoContas {
    public static void main(String[] args) {
        // Aula 07: primeira classe de dominio.
        Conta conta = new Conta(1, "Ana", 1000.00);
        conta.depositar(200.00);
        conta.sacar(50.00);
        System.out.println(conta.resumo());
    }
}
```

Leitura técnica orientada

Percorra o código com estes checkpoints de revisão:

- Estrutura de domínio: identificar onde a entidade de negócio concentra dados.
- Regra de decisão: mapear condições que aprovam ou negam operações.
- Saída observável: conferir mensagens que comprovam sucesso, erro e bloqueio de regra.

Decisões de projeto dos métodos

1. `sacar` sem `else` na classe O padrão de retorno no final já cobre o caso negado. Isso reduz blocos aninhados e mantém o método curto.
2. Regras de `depositar` dentro do objeto O método pode impor regras de domínio, por exemplo:
 - bloquear valor não positivo;
 - bloquear valor acima de um teto por transação;
 - bloquear operação quando conta estiver inativa (em versões futuras).
3. Assinatura como contrato Se a assinatura de `depositar(double valor)` estiver correta, a chamada `conta.depositar(200.00)` continua estável mesmo quando regras internas evoluem.
4. Onde ficou a passagem de parâmetros? Ela não some totalmente: muda de nível.
 - no procedural, passamos quase todo o estado em cada chamada;
 - aqui, passamos só o dado da operação (`valor`) porque o estado já está dentro da conta.

Esse detalhe é o coração da mudança de paradigma.

5. Efeito colateral positivo e risco novo
 - Positivo: menor chance de erro por combinação errada de parâmetros estruturais.
 - Risco novo: se a regra dentro da classe estiver mal definida, várias chamadas repetirão o mesmo erro.

Conclusão de engenharia:

- centralizar regra aumenta consistência;
- também aumenta responsabilidade de testar bem cada método da classe.

Discussão de contrato de retorno

Quando o método retorna `boolean`, o contrato precisa ser tratado como regra de negócio:

- `true` significa operação aprovada e estado atualizado;
- `false` significa operação negada e estado preservado.

Risco de lógica silenciosa:

- ignorar o `false` no chamador cria fluxo aparentemente correto, mas operacionalmente incompleto.

Responsabilidade de quem chama:

- sempre tratar os dois ramos (`true` e `false`) com mensagens e ações coerentes;
- registrar motivo de negativa quando possível (conta inativa, valor inválido, limite excedido);
- evitar seguir o fluxo como se a operação tivesse ocorrido.

Variações para debate técnico:

1. Em quais cenários `boolean` basta?
2. Quando o time precisa de retorno com motivo da falha?
3. Qual custo de não tratar o `false` em produção?

Comparativo de paradigma: procedural vs POO

Diferença que precisa ficar nítida neste ponto da trilha:

- Procedural: o estado é passado entre funções por parâmetros.
- POO: o estado pertence ao objeto, e os métodos operam nesse estado interno.

Impacto no desenho do algoritmo:

- no procedural, o algoritmo coordena dados externos e funções;
- na POO, o algoritmo delega comportamento para objetos com responsabilidades claras.

Critério de maturidade técnica:

- reconhecer quando o custo de passar muitos parâmetros indica necessidade de modelagem por objeto.

Perguntas de decisão

1. O que ficou dentro da classe?
2. Qual vantagem de `conta.depositar(...)`?
3. Qual papel do construtor?
4. Em que casos `sacar` com `else` explícito pode melhorar leitura?
5. Quais regras você incluiria em `depositar` sem mudar assinatura?
6. O que esta aula resolve que o modelo por arrays não resolvia bem?
7. Em que ponto passar muitos parâmetros deixa de ser detalhe e vira problema de modelagem?
8. Se `conta` é uma unidade de domínio, o que não deveria mais ficar no fluxo principal?
9. Quais partes do algoritmo passaram de "coordenação estrutural" para "intenção de negócio"?

Variações de cenário

1. Criar segunda conta e operar nela.
2. Testar saque negado em uma das contas.
3. Simular erro humano: trocar referência de conta e observar impacto.

Exercícios progressivos

1. Adicionar método `mostrarTitular`.
2. Ajustar `resumo` para incluir formato monetário.
3. Escrever duas linhas comparando legibilidade entre chamada procedural por índice e chamada por instância.
4. Listar, em tabela simples, o que o chamador precisava saber antes e o que precisa saber agora.
5. Criar uma versão de `depositar` com validação extra e explicar por que a mudança não altera chamadas externas.

Desafio de equipe

Criar método `transferirPara(Conta destino, double valor)`.

Critérios de aceite dos exercícios

- Regra principal continua válida após alterações.
- Cenário de erro foi testado e tratado sem quebrar execução.
- Código permanece legível e coerente com o nível da aula.

Checklist de progressividade técnica

- Pré-requisito confirmado: leitura e execução do código-base da aula anterior.
- Novidade técnica identificada: explicar em uma frase o recurso novo desta aula.
- Complexidade controlada: apontar qual decisão de projeto evita acoplamento desnecessário.
- Evidência prática: executar ao menos um caso de sucesso e um caso de erro no Tryit.

Fechamento executivo

O ganho desta aula não é apenas sintaxe nova, é redução de ambiguidade estrutural. Agora dados e comportamento estão juntos na mesma unidade de domínio.

Na próxima, vamos escalar essa unidade para operar múltiplas contas com consistência.

Revisão rápida (5 minutos)

1. O que mudou objetivamente da aula 06 para a aula 07?
2. Qual risco estrutural foi reduzido com a conta como unidade?
3. Onde a passagem de parâmetros diminuiu e por quê?
4. Qual parte da regra de negócio saiu do fluxo principal e entrou na classe?
5. Evolução: qual pequena extensão agregaria mais valor sem aumentar acoplamento?

Mini-missão de fechamento (5 min):

- Execute um cenário de borda no Tryit e justifique o resultado em 3 linhas.

Execução no W3Schools Tryit

1. Abrir o compilador online: https://www.w3schools.com/java/tryjava.asp?filename=demo_compiler
2. Colar todo o conteúdo de AppGestaoContas.java no editor.
3. Clicar em Run, registrar saída base e depois testar variações e exercícios.
4. Manter uma aba separada para cada exercício e comparar resultados.

Referências W3Schools da aula

- https://www.w3schools.com/java/java_classes.asp
- https://www.w3schools.com/java/java_class_methods.asp
- https://www.w3schools.com/java/java_compiler.asp

Leituras complementares

- <https://docs.oracle.com/javase/tutorial/>
- <https://dev.java/learn/>